✦ NOVA ✦

Token Specification · v0.1

Formal token rules for the NOVA lexer

Reference document for compiler/lexer and compiler/parser phases

01 Overview

This document is the authoritative reference for every token the NOVA lexer produces. It defines: the token type name, its C enum value, the exact lexical rule (as a regular expression or prose rule), example lexemes, and the precedence of operators.

The lexer is implemented in C (compiler/lexer/src/lexer.c). This spec drives that implementation and any future reimplementations (e.g. a Rust port for the LSP). If there is a conflict between this document and the code, this document is authoritative.

> *Token types are defined in compiler/lexer/include/token.h as the NovaTokenType enum. The order of entries in that enum must match the TOKEN_NAMES array in token.c.*

02 Source Encoding & Comments

NOVA source files are UTF-8 encoded. The lexer is byte-oriented but is Unicode-aware in the following specific cases:

→ (U+2192, bytes: E2 86 92) — accepted as an alias for the ASCII arrow ->
² (U+00B2, bytes: C2 B2) — valid inside unit expressions [ ]
³ (U+00B3, bytes: C2 B3) — valid inside unit expressions [ ]
· (U+00B7, bytes: C2 B7) — middle dot, valid in unit expressions
⊙ (U+2609, bytes: E2 98 89) — solar symbol, valid in unit identifiers

All other multi-byte UTF-8 sequences in identifier position are accepted verbatim (pass-through). Multi-byte sequences outside identifier or unit position produce a TOK_ERROR.

Comments

NOVA has one comment form: a line comment beginning with --. Everything from -- to the end of the line (exclusive of the newline) is discarded by the lexer.

```
-- this is a comment
let x = 42  -- inline comment after code
```

> *There are no block comments in NOVA. This is a deliberate design choice: block comments nest badly and make tooling harder. Line comments are sufficient.*

03 Whitespace and Newlines

Spaces, tabs, and carriage returns are skipped silently between tokens. Newlines (U+000A, LF) produce a TOK_NEWLINE token. The parser uses TOK_NEWLINE for statement termination (NOVA does not require semicolons).

> *The lexer does NOT implement automatic semicolon insertion. The parser decides which newlines are significant. TOK_NEWLINE is always emitted — never suppressed by the lexer.*

04 Literal Tokens

Token type	Enum	Regex / Rule	Example lexemes
TOK_INT_LIT	INT_LIT	[0-9][0-9_]* | 0x[0-9a-fA-F_]+ | 0b[01_]+	42 1_000 0xFF 0b1010
TOK_FLOAT_LIT	FLOAT_LIT	[0-9][0-9_]* '.' [0-9]* exponent? | [0-9]+ exp	3.14 1.0e10 6.674e-11 1.989e30
TOK_UNIT_LIT	UNIT_LIT	(int | float) '[' unit_expr ']'	9.8[m/s] 845000.0[N] 1.0[kg]
TOK_STRING_LIT	STRING_LIT	'"' (char | escape | interp)* '"'	"Hello" "v = {v:.2}"
TOK_BOOL_LIT	BOOL_LIT	true | false	true false

Integer literals

Decimal: one or more decimal digits, underscores allowed as visual separators (ignored). Hex: 0x prefix, hex digits and underscores. Binary: 0b prefix, 0/1 and underscores.

```
42        -- decimal
1_000_000 -- with separators
0xFF      -- hex
0b1010    -- binary
```

Float literals

Integer part, optional decimal part, optional exponent. Underscores allowed in digit runs. Exponent: e or E followed by optional +/- and decimal digits.

```
3.14
6.674e-11   -- gravitational constant
1.989e30    -- solar mass
1.0e-9      -- nano
```

Unit literals

A numeric literal (integer or float) immediately followed by [begins a unit literal. The lexer scans until the matching], producing a single TOK_UNIT_LIT token whose lexeme spans the entire "value[unit]" sequence. The unit expression may contain: ASCII letters and digits, /, *, ^, ., spaces, and valid UTF-8 unit characters.

```
9.80665[m/s]          -- acceleration
845000.0[N]           -- force
6.674e-11[N*m^2/kg^2] -- gravitational constant with units
1.0[M_sun]            -- custom stellar mass unit
```

> *The unit expression is not parsed by the lexer — it is handed as a raw string to the unit resolver (Phase 2, Rust). The lexer only ensures the brackets are balanced.*

String literals

Delimited by double quotes. Escape sequences: \n (newline), \t (tab), \\ (backslash), \" (quote). Interpolation placeholders {expr} are included verbatim in the string token; the parser extracts them.

```
"Hello, universe!"
"r = {r:.4}  v = {v as [km/s]:.2} km/s"
"mass = {mass} kg"
```

05 Keywords

Keywords are reserved identifiers. The lexer resolves them during identifier scanning: if a scanned identifier matches a keyword exactly, it produces the keyword token instead of TOK_IDENT.

Declaration keywords

Keyword	Enum	Role	Example
`mission`	`TOK_MISSION`	Define a function/procedure	mission delta_v(...) -> Float[m/s]
`parallel`	`TOK_PARALLEL`	Mark a mission as auto-parallelised	parallel mission reduce(...)
`constellation`	`TOK_CONSTELLATION`	Define a module	constellation Orbital { ... }
`absorb`	`TOK_ABSORB`	Import names from a constellation	absorb cosmos.stats.{ pearson }
`model`	`TOK_MODEL`	Define a neural network architecture	model CosmoTransformer { ... }
`layer`	`TOK_LAYER`	Define a layer inside a model	layer dense(512, activation=.relu)
`struct`	`TOK_STRUCT`	Define a record type	struct Star { mass: Float[kg] }
`enum`	`TOK_ENUM`	Define an algebraic data type	enum Signal { Some(T), None }
`trait`	`TOK_TRAIT`	Define a shared behaviour contract (Rust-style)	trait Measurable { ... }
`interface`	`TOK_INTERFACE`	Define a module-level contract (Java-style)	interface Optimiser { ... }
`implements`	`TOK_IMPLEMENTS`	Declare that a struct satisfies an interface	struct AdamW implements Optimiser
`extends`	`TOK_EXTENDS`	Single-inheritance for structs	struct B extends A { ... }
`export`	`TOK_EXPORT`	Make a name visible outside the constellation	export mission period(...)
`extern`	`TOK_EXTERN`	Declare an external (FFI) symbol	extern "C" mission malloc(...)
`unit`	`TOK_UNIT_KW`	Declare a custom unit alias	unit parsec = 3.086e16[m]

Binding keywords

Keyword	Enum	Role	Example
`let`	`TOK_LET`	Immutable binding — value cannot be reassigned	let mass = 5.972e24[kg]
`var`	`TOK_VAR`	Mutable binding — value can be reassigned	var altitude = 400.0[km]

Control flow keywords

Keyword	Enum	Role	Example
`if`	`TOK_IF`	Conditional branch	if snr > 10.0 { ... }
`else`	`TOK_ELSE`	Alternative branch	else { ... }
`for`	`TOK_FOR`	Iterator loop	for batch in data.batches(8)
`in`	`TOK_IN`	Range/iterator keyword inside for	for x in 0..100
`while`	`TOK_WHILE`	Condition loop	while running { ... }
`loop`	`TOK_LOOP`	Infinite loop (exit via break)	loop { if done { break } }
`break`	`TOK_BREAK`	Exit current loop	break
`continue`	`TOK_CONTINUE`	Skip to next iteration	continue
`return`	`TOK_RETURN`	Return a value from a mission	return isp * g0 * ln(ratio)
`match`	`TOK_MATCH`	Exhaustive pattern matching	match result { Ok(v) => ... }

AI / scientific keywords

Keyword	Enum	Role	Example
`autodiff`	`TOK_AUTODIFF`	Begin automatic differentiation block	autodiff(loss) { net.update(...) }
`gradient`	`TOK_GRADIENT`	Explicit gradient expression	let g = gradient(loss, wrt params)
`wrt`	`TOK_WRT`	With-respect-to (used after gradient)	gradient(loss, wrt net.params)
`pipeline`	`TOK_PIPELINE`	Ordered transform pipeline block	|> pipeline [filter(...), map(...)]
`on`	`TOK_ON`	Device placement block prefix	on device(gpu) { ... }
`device`	`TOK_DEVICE`	Device specifier (used after on)	on device(gpu)
`wave`	`TOK_WAVE`	Lazy stream / sequence type keyword	let stream: Wave = ...

Output, safety, and type keywords

Keyword	Enum	Role	Example
`transmit`	`TOK_TRANSMIT`	Print / emit output	transmit("Hello, universe!")
`panic`	`TOK_PANIC`	Abort with a message (unrecoverable error)	panic("unreachable")
`unsafe`	`TOK_UNSAFE`	Begin unsafe block (raw pointers allowed)	unsafe { *ptr = 42 }
`raw`	`TOK_RAW`	Begin raw memory block (manual allocation)	raw { let p = alloc(64) }
`as`	`TOK_AS`	Unit conversion or type cast	v as [km/s]
`Void`	`TOK_VOID`	Return type for missions that return nothing	mission main() -> Void
`Never`	`TOK_NEVER`	Return type for missions that never return	mission abort() -> Never
`self`	`TOK_SELF_KW`	Reference to the current struct instance	mission measure(self) -> Float[kg]

06 Operators

Operator precedence follows standard mathematical convention. Higher precedence number = binds tighter. Operators at the same level are left-associative unless noted.

Operator	Enum	ASCII / Unicode	Precedence	Meaning
`=`	TOK_EQ	=	1 (right-assoc)	Assignment
`+=`	TOK_PLUS_EQ	+=	1	Add-assign
`-=`	TOK_MINUS_EQ	-=	1	Sub-assign
`*=`	TOK_STAR_EQ	*=	1	Mul-assign
`/=`	TOK_SLASH_EQ	/=	1	Div-assign
`|>`	TOK_PIPE_GT	|>	2	Pipe: pass left into right
`..`	TOK_DOT_DOT	..	3	Integer range (exclusive end)
`||`	TOK_OR	||	4	Logical OR
`&&`	TOK_AND	&&	5	Logical AND
`==`	TOK_EQ_EQ	==	6	Equal
`!=`	TOK_BANG_EQ	!=	6	Not equal
`<`	TOK_LT	<	7	Less than
`>`	TOK_GT	>	7	Greater than
`<=`	TOK_LT_EQ	<=	7	Less or equal
`>=`	TOK_GT_EQ	>=	7	Greater or equal
`+`	TOK_PLUS	+	8	Addition (unit-checked)
`-`	TOK_MINUS	-	8	Subtraction (unit-checked)
`*`	TOK_STAR	*	9	Multiplication (unit-aware)
`/`	TOK_SLASH	/	9	Division (unit-aware)
`%`	TOK_PERCENT	%	9	Modulo
`@`	TOK_AT	@	10	Tensor matrix multiplication
`^`	TOK_CARET	^	11 (right-assoc)	Power, unit-aware: [m]^2 -> [m2]
`!`	TOK_BANG	!	12 (unary)	Logical NOT
`-`	TOK_MINUS	-	12 (unary)	Negation
`->`	TOK_ARROW	-> or →	n/a (syntax)	Return type arrow
`=>`	TOK_FAT_ARROW	=>	n/a (syntax)	Match arm / lambda body
`?`	TOK_QUESTION	?	n/a (postfix)	Propagate Option/Result
`&`	TOK_AMP	&	n/a (prefix)	Immutable borrow

07 Delimiters

Token type	Enum	Regex / Rule	Example lexemes
TOK_LPAREN	LPAREN	'('	(— function call, grouping
TOK_RPAREN	RPAREN	')'	)
TOK_LBRACE	LBRACE	'{'	{ — block, struct literal

Token type	Enum	Regex / Rule	Example lexemes
TOK_RBRACE	RBRACE	'}'	}
TOK_LBRACKET	LBRACKET	'['	[— array index, unit suffix start
TOK_RBRACKET	RBRACKET	']'	]
TOK_COMMA	COMMA	','	, — argument / element separator
TOK_SEMICOLON	SEMICOLON	';'	; — explicit statement separator (optional)
TOK_COLON	COLON	':'	: — type annotation
TOK_COLON_COLON	COLON_COLON	'::'	:: — path separator (cosmos::stats)
TOK_DOT	DOT	'.'	. — field access, method call
TOK_HASH	HASH	'#'	# — attribute annotation
TOK_PIPE	PIPE	'|'	| — enum variant separator / borrow
TOK_AMP	AMP	'&'	& — borrow operator

08 Special Tokens

Token type	Enum	Regex / Rule	Example lexemes
TOK_IDENT	IDENT	[a-zA-Z_][a-zA-Z0-9_]* (and Unicode letters)	mass delta_v CosmoTransformer _tmp
TOK_NEWLINE	NEWLINE	U+000A (LF)	end of each source line
TOK_EOF	EOF	end of input	always the last token
TOK_ERROR	ERROR	any unrecognised byte	$ ` ~ (outside strings)

> *TOK_IDENT is produced for any identifier that does not match a keyword. Keyword lookup is case-sensitive: 'Void' is a keyword, 'void' is an identifier.*

09 Lexer Rules Summary (EBNF)

The following EBNF grammar summarises the lexical structure of NOVA source. This is the formal input to any future lexer generator.

```
token        ::= literal | keyword | ident | operator | delimiter | NEWLINE | EOF

literal      ::= int_lit | float_lit | unit_lit | string_lit | bool_lit
int_lit      ::= decimal | hex_lit | bin_lit
decimal      ::= DIGIT (DIGIT | '_')*
hex_lit      ::= '0x' (HEXDIGIT | '_')+
bin_lit      ::= '0b' ('0' | '1' | '_')+
float_lit    ::= decimal '.' decimal? exponent?  |  decimal exponent
exponent     ::= ('e' | 'E') ('+' | '-')? DIGIT+
unit_lit     ::= (int_lit | float_lit) '[' unit_expr ']'
unit_expr    ::= (ASCII_LETTER | DIGIT | '/' | '*' | '^' | '.' | UNICODE_UNIT)*
string_lit   ::= '"' (str_char | escape | interp)* '"'
str_char     ::= any UTF-8 byte except '"' and '\'
escape       ::= '\' ('n' | 't' | '\' | '"')
```

```
interp        ::= '{' (any char except '}')+ '}'
bool_lit      ::= 'true' | 'false'

ident         ::= (ALPHA | '_') (ALPHA | DIGIT | '_' | UNICODE_LETTER)*
keyword       ::= 'mission' | 'let' | 'var' | ...  (see Section 05)

comment       ::= '--' (any char except LF)*
whitespace    ::= (' ' | '\t' | '\r')+      -- silently skipped
NEWLINE       ::= '\n'                        -- produces TOK_NEWLINE
```